

Experiment 01

Student Name: Manan Verma

UID:24BAI70008

Branch:CSE-AML

Section/Group:24AIT_KRG-1_B

Semester:5

Subject Name:ADBMS

Subject Code: 24CSP-310

Date:10/7/26

[A.]

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. "Low Salary": All monthly salaries strictly less than \$20,000.
2. "Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. "High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

Output Table

category	accounts_count
Low Salary	1
Average Salary	1
High Salary	3

Ans.

SELECT category, COUNT(a.account_id) AS accounts_count

```

FROM (
    VALUES
        ('Low Salary'),
        ('Average Salary'),
        ('High Salary')
) AS c(category)
LEFT JOIN Accounts a
ON (
    (c.category = 'Low Salary' AND a.income < 20000)
    OR
    (c.category = 'Average Salary' AND a.income BETWEEN 20000 AND 50000)
    OR
    (c.category = 'High Salary' AND a.income > 50000)
)
GROUP BY category
ORDER BY
CASE
    WHEN category = 'Low Salary' THEN 1
    WHEN category = 'Average Salary' THEN 2
    ELSE 3
END;

```

[B.]

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.

2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

Output Table:

name	dept_name	salary
Alice	HR	50000
Bob	HR	25000
Charlie	IT	80000
David	IT	40000
Emma	Sales	45000

Ans.

```
SELECT e.emp_id, e.emp_name, d.dept_name, s.salary
FROM Employees e
JOIN Departments d ON e.dept_id=d.dept_id
JOIN Salaries s ON e.emp_id=s.emp_id;
```

```
UPDATE Salaries
SET salary=salary*1.10
WHERE emp_id IN (
    SELECT e.emp_id
    FROM Employees e JOIN Departments d ON e.dept_id=d.dept_id
    WHERE d.dept_name='HR'
);
```

```
DELETE FROM Salaries WHERE salary<30000;
```

```
SELECT e.emp_id, e.emp_name, s.salary
FROM Employees e
JOIN Salaries s ON e.emp_id=s.emp_id
WHERE s.salary>(SELECT AVG(salary) FROM Salaries);
```

[C.]

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called `pizza_toppings`.

Every customer must choose exactly 3 different toppings to prepare a pizza.

Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements

Every pizza must contain exactly 3 different toppings.

The same topping cannot appear more than once in a pizza.

Different arrangements of the same toppings should be considered the same pizza.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only one should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

First by highest total cost

If two pizzas have the same cost, sort them alphabetically.

Output Table	
pizza	total_cost
Chicken,Pepperoni,Sausage	1.75
Extra Cheese,Pepperoni,Sausage	1.60
Chicken,Extra Cheese,Sausage	1.65
Extra Cheese,Onions,Sausage	1.35
Chicken,Onions,Sausage	1.50
Chicken,Extra Cheese,Pepperoni	1.45
Extra Cheese,Onions,Pepperoni	1.15
Chicken,Onions,Pepperoni	1.30
Chicken,Extra Cheese,Onions	1.20
Extra Cheese,Onions,Sausage	1.35

Ans.

```
SELECT p1.topping_name||', '||p2.topping_name||', '||p3.topping_name AS pizza,
p1.ingredient_cost+p2.ingredient_cost+p3.ingredient_cost AS total_cost
FROM pizza_toppings p1
JOIN pizza_toppings p2 ON p1.topping_name<p2.topping_name
JOIN pizza_toppings p3 ON p2.topping_name<p3.topping_name
ORDER BY total_cost DESC, pizza;
```

[D.]

Amazon wants to identify returning customers who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements

1. Compare purchases made by the same user only.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur after the first purchase.
4. Same-day purchases must not be considered.

5. The second purchase must occur within 7 days of the first purchase.

6. Display only the unique User IDs that satisfy the above conditions.

Sort the output in ascending order of user_id.

Ans.

```
SELECT DISTINCT p1.user_id
FROM purchases p1
JOIN purchases p2
ON p1.user_id=p2.user_id
AND p2.purchase_date>p1.purchase_date
AND p2.purchase_date<=p1.purchase_date+INTERVAL '7 days'
ORDER BY user_id;
```